

PROGRAMMATION ORIENTEE OBJET

ESSENTIEL

La programmation orientée objet (POO) est une manière de programmer où l'on organise le code autour d'objets plutôt que de simples fonctions ou instructions. C'est un concept central en informatique et en génie logiciel.

1- Idée de base

Au lieu de voir un programme comme une suite d'instructions, on le voit comme un ensemble d'objets qui interagissent entre eux.

Un objet représente quelque chose du monde réel ou abstrait : Une voiture, Un étudiant, Un compte bancaire

2- Classe et objet

Classe : modèle ou plan (comme un moule)

Objet : instance concrète créée à partir de la classe

Exemple :

Classe : voiture

Objet : Toyota, Mercédès, etc...

3- Les 4 piliers de la POO

a- Encapsulation

C'est le fait de cacher les détails internes d'un objet et contrôler l'accès aux données. Souvenons nous dans les programmes écrits jusqu'à maintenant, les variables/types de données et de traitement de ces données étaient séparées

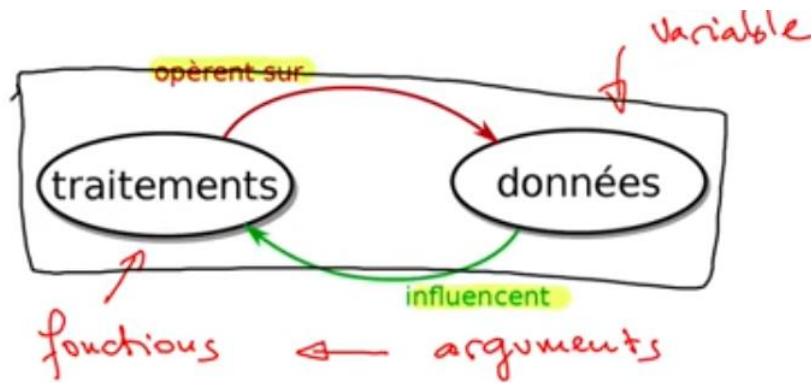


Figure 1: programmation impérative/procédurale (rappel)

Ainsi l'on regroupe dans le même objet informatique (« concept ») les données et les traitements qui lui sont spécifiques :

Attributs : les données incluses dans un objet

Méthodes : les fonctions (=traitements) définies dans un objet

Exemple 1 : un compte bancaire ne permet pas de modifier directement le solde, mais via des méthodes comme `deposer()` ou `retirer()`.

Exemple 2 : calcul de la surface d'un rectangle

`double` surface (`double` largeur, `double` hauteur) ;

`int` main ()

{

`double` largeur (3.0) ;

`double` hauteur (4.0) ;

 Cout « "la surface du rectangle est : "

 « surface (largeur, hauteur) » endl ;

`return` 0 ;

}

`double` surface (`double` largeur, `double` hauteur)

{

`return` largeur * hauteur ;

}

b- Abstraction

Consiste à montrer seulement l'essentiel et cacher la complexité. Pour être véritablement intéressant, un objet doit permettre un certain degré d'**abstraction**.

Le processus d'abstraction consiste à identifier pour un ensemble d'éléments :

- Des caractéristiques communes à tous les éléments
- Des mécanismes communs à tous les éléments

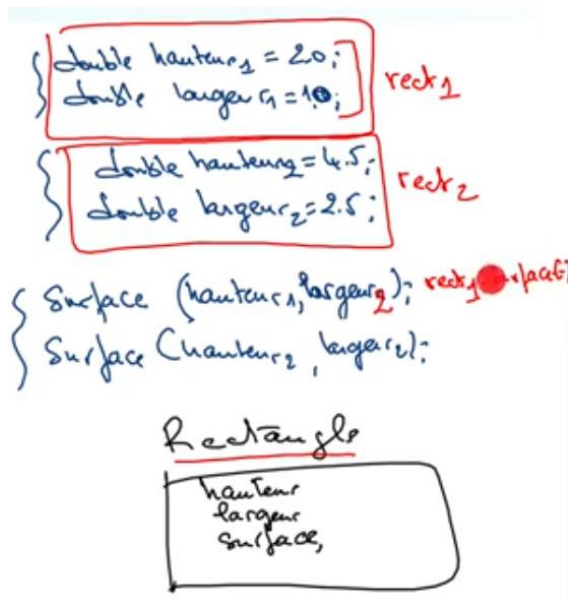


Figure 2: principe d'abstraction

Exemple : l'interface d'une voiture

Volant, accélérateur, pédale de frein etc.

Tout ce qu'il faut savoir pour la conduire (mais pas la réparer ! ni comprendre comment ça marche)

L'interface ne change pas, même si l'on change de moteur...et même si on change de voiture (dans une certaine mesure) : abstraction de la notion de voiture (en tant qu'» objet à conduire »)

Remarque : en plus du regroupement des données et des traitements relatifs à une entité, l'**encapsulation** permet en effet de définir *deux niveaux* de perception des objets :

- Niveau externe : partie visible (par les programmeurs-utilisateurs) :
 - L'interface : *entête* de quelques méthodes bien choisies (résultat du processus d'abstraction)
- Niveau interne : détails d'implémentation
 - Méthodes et attributs accessibles uniquement depuis l'intérieur de l'objet (ou d'objets similaires)
 - Définition de toutes les méthodes de l'objet

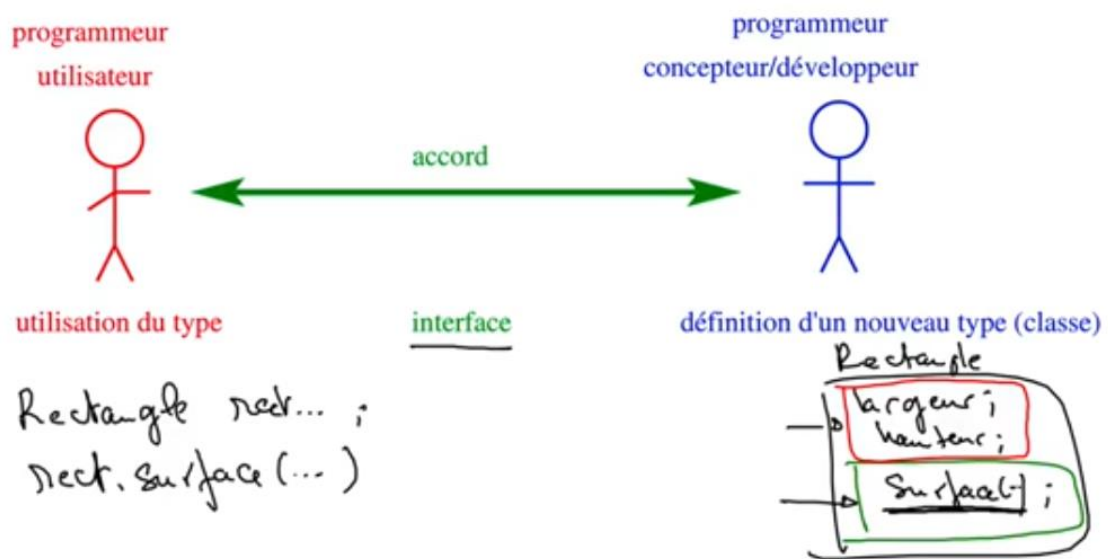


Figure 3: Encapsulation et interface

Il y'a donc deux facettes à l'encapsulation :

- Regroupement de tout ce qui caractérise l'objet : données (attributs) et traitements (méthodes)
- Isolement et dissimulation des détails d'implémentation
- Interface = ce que le programmeur-utilisateur (hors de l'objet) peut utiliser en se concentrant sur les attributs et les méthodes concernant l'objet (abstraction).

```
double largeur(3.0);
double hauteur(4.0);

cout << "Surface : "
    << surface(largeur, hauteur)
    << endl;
```

```
Rectangle rect(3.0, 4.0);
cout << "Surface : "
    << rect.surface()
    << endl;
```

Figure 4: encapsulation rectangle

L'intérêt de regrouper les traitements et les données conceptuellement reliées est de permettre une meilleure visibilité et une meilleure cohérence au programme, d'offrir une plus grande modularité.

L'intérêt de séparer les niveaux *interne et externe* est de donner un **cadre plus rigoureux** à l'utilisation des objets utilisés dans un programme. Les objets ne peuvent être utilisés qu'au travers de leurs interfaces (*niveau externe*) et donc les éventuelles modifications de la structure interne restent **invisibles** à l'extérieur.

Règle : *les attributs d'un objet ne doivent pas être accessibles depuis l'extérieur, mais uniquement par des méthodes.*

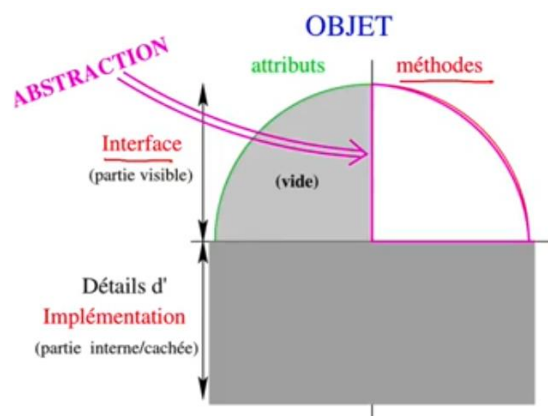
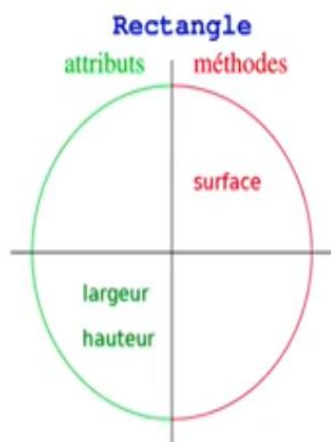


Figure 5: objet

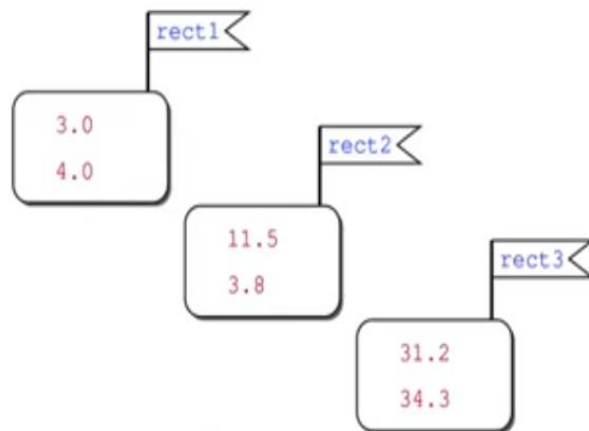
En programmation Objet :

- Le résultat des processus d'encapsulation et d'abstraction s'appelle une **classe**, ce qui représente une catégorie d'objets.

- Une classe définit un **type**
- Une réalisation particulière d'une classe s'appelle une **instance** ou encore un **objet**.



classe
type (abstraction)
existence conceptuelle
(écriture du programme)



objets/instances
variables en mémoire
existence concrète
(exécution du programme)

c- Héritage

Permet de créer une classe à partir d'une autre.

Exemple :

Classe mère : véhicule

Classe fille : voiture, Moto

La voiture hérite des caractéristiques du véhicule.

d- Polymorphisme

Permet d'utiliser une même méthode avec des comportements différents.

Exemple : une méthode `déplacer()` :

Pour une voiture -> roule

Pour un avion -> vole

Exemple : Tu utilises un téléphone sans connaître tous les circuits internes.

1- Exemple simple en Python

```
Class Voiture :
    def __init__(self, marque, vitesse) :
        self.marque = marque
        self.vitesse = vitesse
    def accelerer(self) :
        self.vitesse += 10
        print(self.marque, "accélère à", self.vitesse)
# Création d'un objet
ma_voiture = Voiture("Toyota", 50)

#Utilisation
ma_voiture.accelerer()
```

2- Pourquoi utiliser la POO

Organisation claire du code

Réutilisation (grâce à l'héritage)

Maintenance facile

Adaptée aux projets complexes (comme les applications, systèmes embarqués, IA, etc.)

3- Où utiliser la POO ?

Développement d'applications (Java, c++, Python)

Jeux vidéos

Systèmes embarqués (Arduino, ESP32)

IA

Application POO : Gestion d'un moteur (cas réel)

Objectif

Créer une classe qui représente un moteur à courant continu (MCC) et simuler son comportement

Travail demandé

Créer une classe Moteur avec :

Attributs :

Nom (ex : Moteur A)

Vitesse (0 au départ)

Etat ("OFF" ou "ON")

Méthodes :

Demarrer() -> met le moteur à ON

Arreter () -> met le moteur à OFF et vitesse = 0

Accelerer (valeur) -> augmenter la vitesse

Afficher_etat () -> affiche toutes les infos

Solution (python) :

`Class moteur :`

```
def _init_(self, nom) :  
    self.nom = nom  
    self.vitesse = 0  
def demarrer(self) :  
    self.etat = "ON"  
    print(self.nom, "est démarré")  
def arreter(self) :  
    self.etat = "OFF"  
    print(self.nom, "est arrêté")  
def accelerer(self, valeur) :  
    if self.etat == "ON"  
        self.vitesse += valeur
```

```

        print("Vitesse", self.vitesse)
    else :
        print("le moteur est éteint")
def afficher_etat(self) :
    print("Nom :", self.nom)
    print("Etat :", self.etat)
    print("Vitesse :", self.vitesse)

# Test
m1 = Moteur("Moteur A")

m1.demarrer()
m1.accelerer(50)
m1.afficher_etat()
m1.arreter()

#Utilisation
ma_voiture.accelerer()

```

Solution (C++) :

```

#include <iostream>
#include <string>
using namespace std ;

Class moteur {
private :
    string nom ;
    int vitesse ;
    bool etat ; // false = OFF, true = ON

public :
    // Constructeur
    Moteur (string n) {
        nom = n ;
        vitesse = 0 ;
        etat = false ;
    }
    void demarrer () {
        etat = true ;
        cout << nom << "est démarré" << endl ;
    }
}

```

```

}
void arreter () {
    etat = false ;
    vitesse = 0 ;
    cout << nom << "est arrêté" << endl ;
}
void accelerer (int valeur) {
    if (etat) {
        vitesse += valeur ;
        cout << nom << "Vitesse :" << vitesse << endl ;
    } else {
        cout << nom << "Le moteur est éteint" << vitesse << endl ;
    }
}

void afficherEtat () {
    cout << "Nom :" << nom << endl ;
    cout << "Etat :" << (etat ? "ON" : "OFF") << endl ;
    cout << "Vitesse :" << vitesse << endl ;
}
};
// Programme principal
int main() {
    Moteur m1("Moteur A");

    m1.demarrer()
    m1.accelerer(50)
    m1.afficherEtat()
    m1.arreter()

    return 0 ;
}

```